

리버싱 프로젝트 3

다층 보호가 적용된 오프라인 라이선스 검증 프로그램 분석 및 인증 우회

정명훈

목차

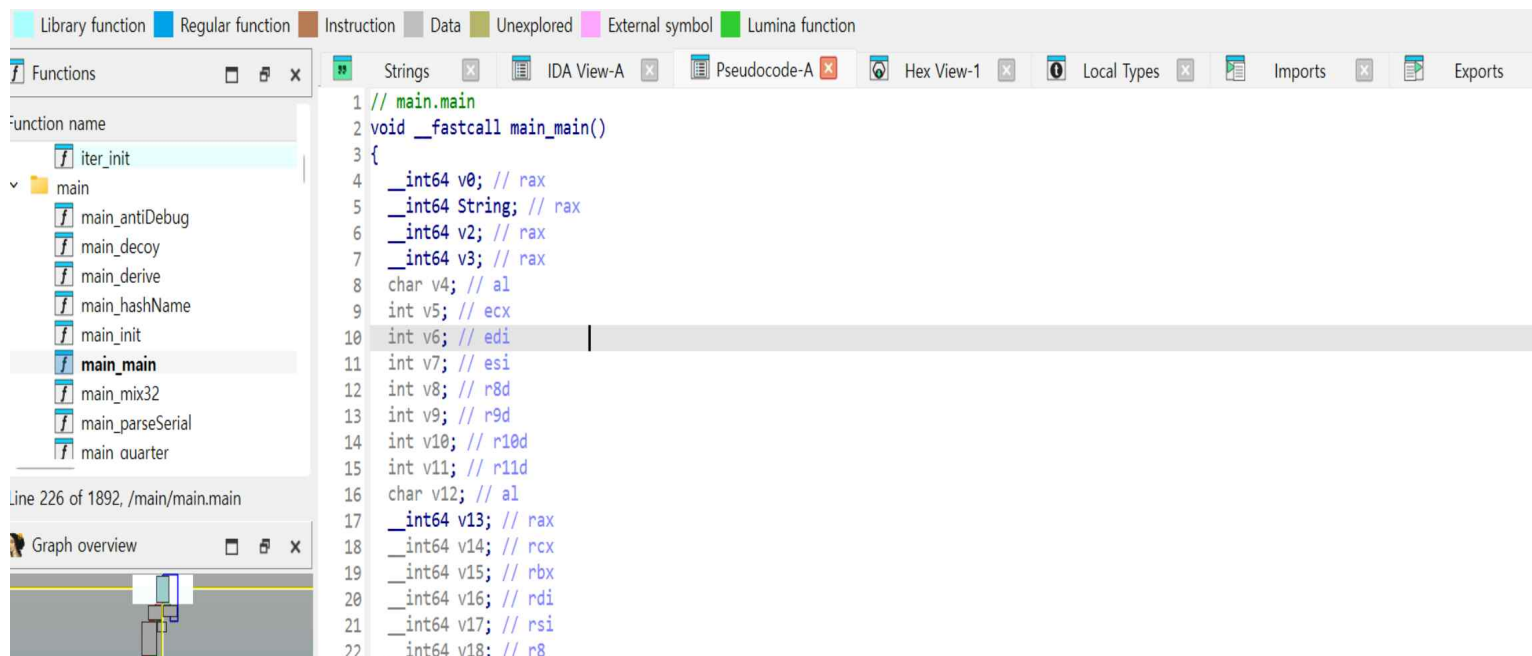
1. 프로그램 환경
2. 정적 분석을 통한 로그인
3. 동적분석을 통한 로그인
4. xdbg64를 이용한 바이너리 패치
5. 배운점
6. 아쉬운 점

1. 프로그램 환경

ida-64-free, visual studio2022, visual studio code, python 3.13(64bit), xdbg64, 실행파일은 go code가 compile된 거다.

2.정적 분석을 통한 로그인

1. 생각보다 gpt가 만든 exe파일을 ida에 올리니 main이 너무 쉽게 찾아지고 또 그 안의 함수들도 이름만 보고 기능을 어느 정도 유추가능하게 나왔다 그래도 너무 쉽겐 보지 말자 우선 이 프로그램이 뭘 하는지를 파악하고 그걸 이용해 역연산이나 브루토포스 방법으로 정답을 찾으면 된다 우선 먼저 string table을 먼저 봤더니 암호화된 것도 있고 내용이 너무 길어서 우선 main함수부터 찾기로 결정하고 function table에서 main을 찾았더니 다행이도 main함수 이름이 암호화가 안 되어 있어서 비교적으로 쉽게 찾을 수 있었다.



전의 if문을 봤더니 놓친 조건이 있길래 그 조건을 한 번 봤는데

```
0  v26 = *(_OWORD *)v25;  
1  if ( LOBYTE(v46._r0) && (unsigned __int64)(r1 - 3) <= 0x15 )  
2  {  
3      if ( main_antiDebug() )  
4      {  
5          *(_OWORD *)v25 = v26;  
6          v14 = main_decoy(r0, r1, v7, v8, v9, v10, v11, v12, v13, v26);  
7      }
```

이는 if (serialFormatValid && userIdLength >= 3 && userIdLength <= 24)와 같은 의미를 가지고 이에 입력으로 같은 v46._r0과 r1값을 파악하기 위해 ida의 메인 부분을 위에서부터 하나씩 분석을 해보니

```
String = bufio__ptr_Reader_ReadString();
```

```
v5 = strings_TrimSpace(String);
```

```
v45 = strings_ToLower(v5);
```

```
r0 = v45._r0;
```

```
r1 = v45._r1;
```

이 부분에서 user_id를 입력으로 받고 그 다음에는

```
v6 = bufio__ptr_Reader_ReadString();
```

```
v46 = main_parseSerial(v6)
```

이 부분에서 라이선스를 입력하고 파싱을 하는 걸 알 수 있다.

그럼으로 LOBYTE(v46._r0) != 0 이 부분의 의미는 라이선스 문자열의 형식이 올바르게 파싱되었다는 의미를 가지고 이에 따라서 라이선스 문자열이 올바른지 확인하는 main_parseSerial함수를 자세히 분석해 보면

1. 앞뒤 공백 제거
2. 모든 '-' 제거
3. 남은 문자열 길이가 32인지 검사
4. 각 문자가 0~9, A~F, a~f인지 검사
5. 문자 2개씩 묶어서 16바이트 생성
6. 성공 시 AL=1, 실패 시 AL=0

와 같은 작업을 하는 라이선스 문자열을 바이트 배열로 바꾸는 전처리 함수로 이

함수가 성공하고 이름의 길이가 3이상 24미만이어야지 형식 오류를 범하지 않는다는 것을 알 수 있다. 따라서 앞으로 이 형식을 지켜서 한 번 다시 xdbg64로 입력을 넣어보았더니

```
C:\Users\정명훈>"D:\Downloads\reversing_project3_package\reversing_project3.exe"
RVP3 License Validation Console
User ID: Tomas
License: aaaaaaaa-bbbbbbbb-cccccccc-dddddddd
ACCESS DENIED
```

이렇게 형식검증을 통과했지만 정답이 아니라 접근이 거부된 상태이다. 다음으로는 이제 id에 따른 올바른 라이선스값을 구해야 한다.

그러기 위해서는 우선 main_derive함수부터 하나씩 분석을 해보자

엄밀하게 비교하기 위해서는 ida에서 디컴파일된 부분과 어셈블리 부분을 비교해가면서 비교를 해야 한다. 우선 인자부분부터 살펴보면 ida에는 인자가 두 개인 것처럼 나오지만

```
----
{
    v15 = main_derive(r0, r1);
    *(_OWORD *)&v25[5] = *(_OWORD *)v25;
    v14 = main_vmCheck(v15, v17, v16, v18, v19, v20, v21, v22, v23, v26, *((_OWORD *)&v26 + 1), v25[0], v25[1]);
}
if ( v14 )
```

실제 어셈블리를 보면 exe파일이 go언어로 작성되었다는 점과

```
mov [arg_10], rax
```

```
mov [arg_18], rbx
```

call main_hashName 어셈블리 코드를 보면 이는

RAX = 이름 문자열 주소

RBX = 이름 문자열 길이 와 같은 의미를 가졌을 확률이 높다. 따라서 실제 인자는 string name하나를 받는다고 생각하면 된다. 그 다음에는

```
v2 = main_hashName();
```

```
v19 = v2;
```

로 `main_hashName`이 이름을 기반으로 32비트 해시를 생성하고, 그 결과를 `v19`에 저장한다. 그 다음에 ida 디컴파일된 코드를 보면

```
v23 = v2 ^ 0x61707865;
```

```
v22 = main_rotl32(v2, 7) ^ 0x3320646E;
```

```
v21 = main_mix32(v19 ^ 0x79622D32u);
```

```
v20 = (-1640531535 * a2) ^ 0x6B206574;
```

과 같이 4개의 초기 32비트 상태값 생성하고 있다. 여기서 생성에 사용한 비트값을 분석하면

`0x61707865` = 65 78 70 61 → "expa"

`0x3320646E` = 6E 64 20 33 → "nd 3"

`0x79622D32` = 32 2D 62 79 → "2-by"

`0x6B206574` = 74 65 20 6B → "te k" 이렇게 나오걸 그걸 모아보면 "expand 32-byte k"로 이는 ChaCha20에서 사용하는 유명한 상수다. 따라서 이 함수는 ChaCha20을 그대로 쓰는 것은 아니지만, ChaCha의 상태 초기화와 quarter-round 구조를 참고해 만든 변형 함수라고 추론해 볼 수 있다 물론 자세한 건 한 번 봐야 안다.

```
for (i = 0; i < 6; i = v28 + 1)
```

```
{
```

```
    v28 = i;
```

```
    main_quarter(&v23, &v22, &v21, &v20);
```

```
    v23 ^= -1515870811 * v28 + 324508639;
```

```
    v21 = main_rotl32(
```

```
        v19 ^ v21,
```

복잡한 회전량 계산

```
);  
  
}
```

그 다음 부분을 보면 6번 동안 반복문을 돌면서 무언가를 하고 있는데 이거의 의미를 c 코드로 정리하면

```
for (int round = 0; round < 6; round++) {  
  
    quarter(&s0, &s1, &s2, &s3);  
  
    s0 ^= round * 0xA5A5A5A5 + 0x13579BDF;  
  
    s2 = ROTL32(  
  
        h ^ s2,  
  
        ((3 * round + 5) % 31) + 1  
  
    );  
}
```

}다음과 같다. 또한 즉 main_quarter는 네 개의 32비트 값을 직접 수정하고 quarter함수는 내부 구조를 보았을 때 기준으로

```
a += b;  
  
d ^= a;  
  
d = ROTL(d, n);  
  
c += d;  
  
b ^= c;  
  
b = ROTL(b, n);
```

대략 이런 식을 실행시키긴다는 거에 유의를 두고 그 함수에 나와 전체 흐름을 코드로 요약하면

```
uint8_t derive[16](string name)
```

```
{

uint32_t h = hashName(name);

uint32_t nameLen = len(name);


uint32_t s0 = h ^ 0x61707865;

uint32_t s1 = ROTL32(h, 7) ^ 0x3320646E;

uint32_t s2 = mix32(h ^ 0x79622D32);

uint32_t s3 = (nameLen * 0x9E3779B1) ^ 0x6B206574;


for (uint32_t round = 0; round < 6; round++) {

    quarter(&s0, &s1, &s2, &s3);


    s0 ^= round * 0xA5A5A5A5 + 0x13579BDF;

    s2 = ROTL32(h ^ s2, 6 + 3 * round);

}


uint32_t words[4];


words[0] = s0 ^ mix32(s2);

words[1] = s1 + ROTL32(s3, 3);

words[2] = s2 ^ ROTL32(s0, 11);

words[3] = s3 + mix32(s1);


uint8_t result[16];
```

```

for (int i = 0; i < 4; i++) {

    result[i * 4 + 0] = words[i];

    result[i * 4 + 1] = words[i] >> 8;

    result[i * 4 + 2] = words[i] >> 16;

    result[i * 4 + 3] = words[i] >> 24;

}

```

```

for (int i = 0; i < 16; i++) {

    result[i] ^= (uint8_t)(29 * i + 83);

    result[i] = ROTL8(result[i], 3);

}

```

```

return result;

```

}다음과 같이 된다. 이제 이것 파이썬으로 만들어보자 파이썬 파일로 따로 만들어 검증해 모았다(프로젝트 폴더에 따로 추가해놨다) 그 다음에는 이제 다음으로 존재하는 vm_check함수를 확인했는데 분석해보니 복잡해 보이지만 어셈블리 코드를 보고 해석을 하면

```

bool vmCheck(uint8_t derived[16], uint8_t serial[16])

```

```

{

    for (int block = 0; block < 4; block++) {

        uint8_t difference = 0;

        for (int i = 0; i < 4; i++) {

            int index = block * 4 + i;

```

```
difference |= derived[index] ^ serial[index];
```

```
}
```

```
if (difference != 0)
```

```
    return false;
```

```
}
```

```
return true;
```

}이와 같이 내가 입력한 serial값과 이름을 기반으로 derive함수에서 생성한 값이 맞는지
를 확인하는 함수다 따라서 이 함수는 derive함수를 잘 구현해서 serial값만 올바르게 구
하면 쉽게 해결할 수 있는 문제다.

```
...
명훈 /OneDrive/Desktop/security_project/proj3_
derive.py
Name: tom
Raw : D95C0C3CF6FA292A86347F354410842D
Serial: D95C0C3C-F6FA292A-86347F35-4410842D
PS C:\Users\정명훈\OneDrive\Desktop\security_
project> █
```

그래서 실제로 위의 파이썬 코드로 쉽게 serial을 구해서 그냥 exe창에 입력하면

```
C:\Users\정명훈>"D:\Downloads\reversing_project3_package\reversing_project3_v2.exe"
RVP3 License Validation Console
User ID: Tom
License: D95C0C3C-F6FA292A-86347F35-4410842D
LICENSE ACCEPTED
```

이렇게 쉽게 뚫린다. 오히려 함수 이름들이 너무 어떤 기능을 하는지 정직하게 나와 있
어서 너무 쉽게 정적분석만으로 해결할 수 있다 그래도 한 번 동적 분석을 통해서 문제
를 해결해 보자

3.동적분석을 통한 로그인

동적분석을 하면

```

00000000106E1C0 48 8B C2 38 mov rdx,qword ptr ss:[rsp+38]
• 00000000106E1D1 E8 EAF4FFFF call reversing_project3_v2.106D6C0
• 00000000106E1D6 0F 10 42 24 movups xmm0,xmmword ptr ss:[rsp]
• 00000000106E1DA 0F 11 42 28 movups xmmword ptr ss:[rsp+28],xmm0
• 00000000106E1DF 0F 10 42 38 movups xmm0,xmmword ptr ss:[rsp+38]
• 00000000106E1E4 0F 11 04 24 movups xmmword ptr ss:[rsp],xmm0

```

이렇게 derive함수 실행 후에 xmm0에 있는값을 확인해 보면

```

XMM0 2D841044357F34862A29FAF63C0C5CD9
XMM1 0000000000000000FF000000000000000
XMM2 323932414636462D4333433043353944
XMM3 0000000000000000FF000000000000000
XMM4 00000000000000000000000000000000
XMM5 00000000000000000000000000000000
XMM6 676E697372657665725C6567616B6361

```

이렇게 나오고 xmm0는 표시 순서가 반대라 이걸 거꾸로 뒤집어 보면D9 5C 0C 3C F6 FA 29 2A 86 34 7F 35 44 10 84 2D가 정답이 된다 이걸 '-'표시를 넣어서 입력하면

```

D:\Downloads\reversing_project3_package\reversing_project3_v2.exe
RVP3 License Validation Console
User ID: Tom
License: D95C0C3C-F6FA292A-86347F35-4410842D
LICENSE ACCEPTED

```

역시 마찬가지로 로그인이 되는 걸 확인할 수 있다. 이렇게 동적분석으로도 정답을 찾았으니 다음은 바이너리 패치로 넘어가자

4.xdbg64를 이용한 바이너리 패치

바이너리 패치도 그냥 쉽다 vm_check함수가 늘 참을 반환하게 하게 패치하면 늘 참이 되서 로그인이 된다. 기존의 vm_check함수의 코드를

```

00000000106DB1E CC int3
00000000106DB1F CC int3
00000000106DB20 48:31C0 xor rax,rax rax:EXTENDED
00000000106DB23 90 nop
00000000106DB24 48:F7D0 not rax rax:EXTENDED
00000000106DB27 90 nop
00000000106DB28 C3 ret
00000000106DB29 90 nop
00000000106DB2A 90 nop
00000000106DB2B 90 nop
00000000106DB2C 90 nop
00000000106DB2D 90 nop
00000000106DB2E 44:0F117C24 20 movups xmmword ptr ss:[rsp+20],xmm15
00000000106DB34 C64424 13 10 mov byte ptr ss:[rsp+13],10
00000000106DB39 C64424 15 20 mov byte ptr ss:[rsp+15],20
00000000106DB3E C64424 17 31 mov byte ptr ss:[rsp+17],31
00000000106DB43 48:BA 5A40100120013: mov rdx,6D3101200110405A
00000000106DB4F 48:895424 18 mov qword ptr ss:[rsp+18],rdx

```

이런식으로 패치해서 실행하면

```

D:\Downloads\reversing_project3_package\reversing_project3_v3.exe
RVP3 License Validation Console
User ID: Tom
License: 00001111-00001111-00001111-00001111
LICENSE ACCEPTED

```

이렇게 엉터리값을 입력해도 로그인인 된다는 걸 확인할 수 있다.

5.배운 점

배운점은 솔직히 많이 없다. project2보다 오히려 어떤 부분에서는 쉬웠던 것 같다 다만 GO라는 낯선언어를 컴파일한 실행파일을 분석하다보니 ida에서 디컴파일 복원이 이전 c에 비해서 많이 어려워지긴 했고 입력받는 부분을 찾는데 꽤나 애먹었으며 암호화하는 과정이 많이 복잡해서 어셈블리와 디컴파일 결과를 비교하면서 복원하는 것도 생각보다 시간이 걸렸다.

6.아쉬운 점

아쉬운 점은 project2와 비교시 디컴파일 내용이 신뢰성이 떨어진다는 점을 제외하면 이전보다 더 단순한 흐름으로 진행되고 여러 보안 프로그램이 암호화 수준은 높지만 따로 따로 놓고 유기적으로 연결되어 있지 않다는 문제점이 있다. 따라서 프로젝트4에서는 보안 요소들이 조금 더 유기적으로 어렵게 연결이 되고 더미 함수와 가짜 검증함수들로 더욱 더 리버싱의 난이도를 올려 진행하려고 한다.

